

AI Usage Report

Assignment: Low-Rank Adaptation (LoRA) Implementation and Application to a Language Model

This report documents the use of generative AI during the LoRA assignment. It is intended to make the process transparent by identifying the AI tool used, the prompts and tasks supported by the tool, the outputs used, and the verification performed before submission.

1. AI Tool Used

Item	Details
AI tool	ChatGPT 5.5 Thinking
Provider	OpenAI
Year	2026
Account type	OpenAI Plus Account
Primary use	Learning support, code explanation, implementation guidance, debugging, and report writing support

2. Purpose of AI Use

Generative AI was used as a support tool to help understand the assignment requirements, structure the implementation, debug errors, and prepare explanations for the report. The AI-generated material was not used as a replacement for understanding the task. The code and explanations were reviewed, executed in Google Colab, and checked before inclusion in the submission.

3. Assignment Components Supported by AI

- Understanding the intuition behind Low-Rank Adaptation and how it reduces trainable parameters.
- Implementing a standalone LoRALayer using two low-rank matrices, `lora_A` and `lora_B`.
- Implementing a non-merged LoRALinear wrapper around an existing `torch.nn.Linear` layer.
- Implementing a merged LoRA linear layer for inference by adding the low-rank update directly to the base weight matrix.
- Verifying mathematical equivalence between merged and non-merged LoRA outputs.
- Preparing the AG News dataset for causal language modeling with padding, truncation, labels, and DataLoaders.
- Loading GPT-Neo-125M as the frozen reference model and creating a second LoRA-adapted copy.
- Replacing selected attention projection layers with LoRA-adapted linear layers.
- Freezing base model parameters and enabling training only for LoRA matrices.
- Inspecting trainable parameters and comparing parameter counts.
- Writing the training loop for the LoRA-adapted model and printing training loss.
- Writing token-level evaluation code that ignores masked padding positions.
- Preparing written explanations and this AI usage report.

4. Summary of Prompts Used

Prompt area	Description of AI request
LoRA concept	Asked for an intuitive explanation of LoRA and how low-rank matrices approximate a full weight update.

Prompt area	Description of AI request
LoRALayer	Asked for help completing a PyTorch LoRALayer class using rank, alpha, dropout, and two trainable matrices.
LoRALinear	Asked for help wrapping an existing nn.Linear layer while freezing the original weights and adding a LoRA update.
Merged LoRA	Asked for help implementing an inference version where the LoRA update is merged into the original weight matrix.
Equivalence test	Asked how to copy LoRA parameters between merged and non-merged models and compare their outputs.
Dataset	Asked for help preparing the AG News dataset for causal language modeling with masked labels.
Model setup	Asked how to load the tokenizer and GPT-Neo-125M model, set padding, device, and evaluation or training mode.
LoRA injection	Asked how to replace attention projection layers such as q_proj, k_proj, v_proj, and out_proj with LoRA wrappers.
Training and evaluation	Asked for a training loop that updates only LoRA parameters and an accuracy function that ignores masked tokens.
Reporting	Asked for help writing explanations and documenting the use of AI according to the course policy.

5. AI-Generated Content Used

The AI-generated responses were used to support the following types of content:

- Code templates for PyTorch modules and functions.
- Short explanations of LoRA formulas and implementation decisions.
- Debugging guidance for errors such as undefined class names and variable naming issues.
- Verification steps for checking trainable parameters, parameter counts, and output equivalence.
- Suggested wording for the final assignment report and AI policy documentation.

6. Edits, Testing, and Verification Performed

The AI-generated material was not inserted without review. The following checks were performed in Google Colab:

- Confirmed that the original linear layer output was recorded as the baseline.
- Confirmed that the LoRA output initially matched the baseline because lora_B was initialized to zero.
- Confirmed that the merged and non-merged LoRA versions produced a maximum absolute difference of 0.0 when using the same parameters.
- Confirmed that the base model parameters were frozen.
- Confirmed that only lora_A.weight and lora_B.weight were trainable after applying LoRA to the language model.
- Confirmed that LoRA was applied to attention projections such as q_proj, k_proj, v_proj, and out_proj.
- Confirmed that the number of trainable parameters in the LoRA model was much smaller than the number required for full fine-tuning.
- Prepared the dataset with labels masked using -100 so padding tokens do not contribute to the loss or token-level accuracy.
- Ran the training loop and printed loss values during training.

7. Parameter Count Summary

The parameter comparison was used to quantify the efficiency benefit of LoRA. The frozen base model was used as a reference, while the LoRA-adapted model contained additional trainable low-rank matrices.

Model	Total parameters	Trainable parameters
Base model	125,198,592	0 as frozen reference; 125,198,592 for full fine-tuning baseline
LoRA-adapted model	125,788,416	589,824

The LoRA model trained 589,824 parameters compared with 125,198,592 parameters for full fine-tuning. This means LoRA trained approximately 0.4711% of the full fine-tuning parameter count and reduced trainable parameters by approximately 99.5289%.

8. Responsibility Statement

I understand that I am responsible for the correctness, accuracy, and originality of the submitted work. Generative AI was used as a support tool for explanation, coding guidance, debugging, and report writing. The code was reviewed, executed, checked, and interpreted before submission. I also understand that using AI does not remove responsibility for ensuring that the submission meets the assignment requirements and does not match another student's work without proper acknowledgement.

9. AI Citation

ChatGPT 5.5 Thinking (2026). Prompt: Assistance with implementing and explaining Low-Rank Adaptation layers, LoRA-adapted linear layers, merged LoRA inference, dataset preparation, LoRA application to GPT-Neo-125M, parameter counting, training, evaluation, debugging, and report writing. Response: Provided explanations, code templates, debugging support, and report-writing assistance. OpenAI Plus Account.

10. Declaration

This report is included to document the use of generative AI in accordance with the course AI policy. The submitted assignment remains my responsibility, and the AI-assisted material was checked and adapted before submission.